Integration Test Design (StreetRace Manager)

High-level integration scenarios (IT-01 .. IT-06)

ID	Scenario	Modules involved	Expected result	Actual result after testing	Errors / logical issues found
IT-01	Assign role without registration	registration + crew_management	Rejects with ValueError	PASS (raises ValueError)	Missing registration guard in role assignment (fixed d2cd6b2)
IT-02	Register driver then enter race	registration + crew_management + inventory + race_management	Race entry stored with correct driver and car	PASS (driver and car stored)	None on happy path
IT-03	Enter race with non-driver	registration + crew_management + inventory + race_management	Rejects with ValueError	PASS (raises ValueError)	Race entry did not enforce driver-only rule (fixed c8b1e1d)
IT-04	Win race updates results, rankings, and cash	race_management + results + rankings + inventory	Outcome win, rankings updated, cash += prize	PASS (all updates applied)	Prize money flow incorrect (fixed 836b178)
IT-05	Lose race damages car; rescue mission	race_management + maintenance + mission_planning +	On loss car condition decreases; rescue requires driver+mec	PASS (damage applied; mission blocked	Car damage + mission-role logic issues (fixed

	requires mechanic	crew_management	hanic available	unless roles available)	73922e1, 27b5be4)
IT-06	Prevent inventory overdraft	inventory	Rejects with ValueError	PASS (raises ValueError)	Overdraft allowed previously (fixed 2b4b3da)

Per-test-case details

Each test case below includes: scenario, modules involved, expected result (what the test asserts), actual result (after testing), and any errors/logical issues detected (with fix commit if applicable).

integration/tests/test_integration_flows.py:

test_enter_race_rejects_non_driver

Scenario: Enter race rejects non driver.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- pytest
- integration.code.race_management
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found:

- Race entry did not enforce driver-only rule.
- Fixed in commit: c8b1e1d

test_inventory_remove_cash_cannot_overdraft

Scenario: Inventory remove cash cannot overdraft.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.inventory
- pytest

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found:

- Inventory allowed overdraft (cash could go negative).
- Fixed in commit: 2b4b3da

test_race_loss_damages_car

Scenario: Race loss damages car.

Why this test is needed: Checks that modules interact correctly for this scenario.

Modules involved:

- integration.code.crew_management
- integration.code.race_management
- integration.code.registration

Expected result:

- Assert: result.outcome == 'lose'
- Assert: mgr.inventory.cars['CAR1'].condition == 80

Actual result after testing: PASS.

Errors / logical issues found:

- Losing a race did not correctly damage the car.
- Fixed in commit: 73922e1

test_race_win_updates_results_rankings_and_cash

Scenario: Race win updates results rankings and cash.

Why this test is needed: Validates end-to-end race flow updates the correct modules consistently.

Modules involved:

- integration.code.crew_management
- integration.code.inventory

- integration.code.race_management
- integration.code.registration

Expected result:

- Assert: result.outcome == 'win'
- Assert: mgr.rankings['Mia'] >= 3
- Assert: inventory.get_cash(mgr.inventory) == before_cash + 500

Actual result after testing: PASS.

Errors / logical issues found:

- Prize money flow was incorrect (cash not credited on win).
- Fixed in commit: 836b178

test_register_driver_then_enter_race_success
Scenario: Register driver then enter race success.

Why this test is needed: Validates end-to-end race flow updates the correct modules consistently.

Modules involved:

- integration.code.crew_management
- integration.code.race_management
- integration.code.registration

Expected result:

- Assert: mgr.races['R1'].driver_name == 'Mia'
- Assert: mgr.races['R1'].car_id == 'CAR1'

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_register_then_assign_role_requires_registration
Scenario: Register then assign role requires registration.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- pytest

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found:

- Missing registration guard in role assignment.
- Fixed in commit: d2cd6b2

test_rescue_mission_requires_driver_and_mechanic_available
Scenario: Rescue mission requires driver and mechanic available.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- integration.code.mission_planning
- pytest
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found:

- Mission start logic allowed starting when only some required roles were available.
- Fixed in commit: 27b5be4

integration/tests/test_integration_iteration2_paths.py

test_assign_members_to_mission_requires_registered_members
Scenario: Assign members to mission requires registered members.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.mission_planning
- pytest

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_assign_role_rejects_invalid_role
Scenario: Assign role rejects invalid role.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- pytest
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_audit_log_records_events_via_manager
Scenario: Audit log records events via manager.

Why this test is needed: Covers an integration decision/edge path to prevent regressions.

Modules involved: (implicit via StreetRaceManager + integration system modules)

Expected result:

- Assert: mgr.events == []
- Assert: len(mgr.events) == 1
- Assert: mgr.events[0].message == 'created manager'

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_create_race_rejects_duplicate_id_and_nonpositive_prize
Scenario: Create race rejects duplicate id and nonpositive prize.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- pytest

- integration.code.race_management

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_damage_and_repair_clamp_and_ignore_nonpositive
Scenario: Damage and repair clamp and ignore nonpositive.

Why this test is needed: Checks that modules interact correctly for this scenario.

Modules involved:

- integration.code.maintenance

Expected result:

- Assert: car.condition == 10
- Assert: car.condition == 0
- Assert: car.condition == 0
- Assert: car.condition == 100

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_enter_race_rejects_missing_race_or_missing_car
Scenario: Enter race rejects missing race or missing car.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- pytest
- integration.code.race_management

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_enter_race_rejects_registered_member_without_driver_role

Scenario: Enter race rejects registered member without driver role.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- pytest
- integration.code.race_management
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_inventory_cash_add_remove_nonpositive_are_noops

Scenario: Inventory cash add remove nonpositive are noops.

Why this test is needed: Protects shared inventory state used by multiple modules (cash must remain valid).

Modules involved:

- integration.code.inventory

Expected result:

- Assert: inventory.get_cash(mgr.inventory) == 0
- Assert: inventory.get_cash(mgr.inventory) == 0

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_inventory_remove_cash_exact_balance_to_zero

Scenario: Inventory remove cash exact balance to zero.

Why this test is needed: Protects shared inventory state used by multiple modules (cash must remain valid).

Modules involved:

- integration.code.inventory

Expected result:

- Assert: inventory.get_cash(mgr.inventory) == 0

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_mission_create_rejects_unknown_type_and_duplicate_id
Scenario: Mission create rejects unknown type and duplicate id.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.mission_planning
- pytest

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_mission_start_respects_role_availability_and_sets_started
Scenario: Mission start respects role availability and sets started.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- integration.code.mission_planning
- pytest

Expected result:

- Raises ValueError
- Assert: mission.started is True

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_register_member_rejects_blank_name
Scenario: Register member rejects blank name.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- pytest
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_register_member_rejects_duplicate_registration
Scenario: Register member rejects duplicate registration.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- pytest
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_register_member_strips_whitespace_and_stores_member
Scenario: Register member strips whitespace and stores member.

Why this test is needed: Checks that modules interact correctly for this scenario.

Modules involved:

- integration.code.registration

Expected result:

- Assert: member.name == 'Mia'
- Assert: 'Mia' in mgr.crew_registry

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_run_race_lose_records_result_updates_rankings_no_prize_and_damages_car
Scenario: Run race lose records result updates rankings no prize and damages car.

Why this test is needed: Validates end-to-end race flow updates the correct modules consistently.

Modules involved:

- integration.code.inventory
- integration.code.race_management

Expected result:

- Assert: result.outcome == 'lose'
- Assert: len(mgr.results) == 1
- Assert: mgr.rankings['Dom'] == 1
- Assert: inventory.get_cash(mgr.inventory) == before_cash
- Assert: mgr.inventory.cars['CAR1'].condition == 80

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_run_race_rejects_missing_race_and_missing_entry
Scenario: Run race rejects missing race and missing entry.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- pytest
- integration.code.race_management

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_run_race_win_records_result_updates_rankings_awards_prize_and_no_damage
Scenario: Run race win records result updates rankings awards prize and no damage.

Why this test is needed: Validates end-to-end race flow updates the correct modules consistently.

Modules involved:

- integration.code.inventory
- integration.code.race_management

Expected result:

- Assert: result.outcome == 'win'
- Assert: len(mgr.results) == 1
- Assert: mgr.rankings['Mia'] == 3
- Assert: inventory.get_cash(mgr.inventory) == before_cash + 250
- Assert: mgr.inventory.cars['CAR1'].condition == before_condition

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

test_set_skill_requires_registration_and_range_checked

Scenario: Set skill requires registration and range checked.

Why this test is needed: Ensures invalid cross-module states are rejected early (prevents inconsistent data flow between modules).

Modules involved:

- integration.code.crew_management
- pytest
- integration.code.registration

Expected result:

- Raises ValueError

Actual result after testing: PASS.

Errors / logical issues found: None detected by this test.

Summary of integration defects detected and fixed

The following defects were detected by the iteration-1 integration flows and then fixed; the current run confirms the fixes.

- test_register_then_assign_role_requires_registration: Missing registration guard in role assignment. (fix commit d2cd6b2)
- test_enter_race_rejects_non_driver: Race entry did not enforce driver-only rule. (fix commit c8b1e1d)
- test_race_win_updates_results_rankings_and_cash: Prize money flow was incorrect (cash not credited on win). (fix commit 836b178)

- test_race_loss_damages_car: Losing a race did not correctly damage the car. (fix commit 73922e1)
- test_rescue_mission_requires_driver_and_mechanic_available: Mission start logic allowed starting when only some required roles were available. (fix commit 27b5be4)
- test_inventory_remove_cash_cannot_overdraft: Inventory allowed overdraft (cash could go negative). (fix commit 2b4b3da)
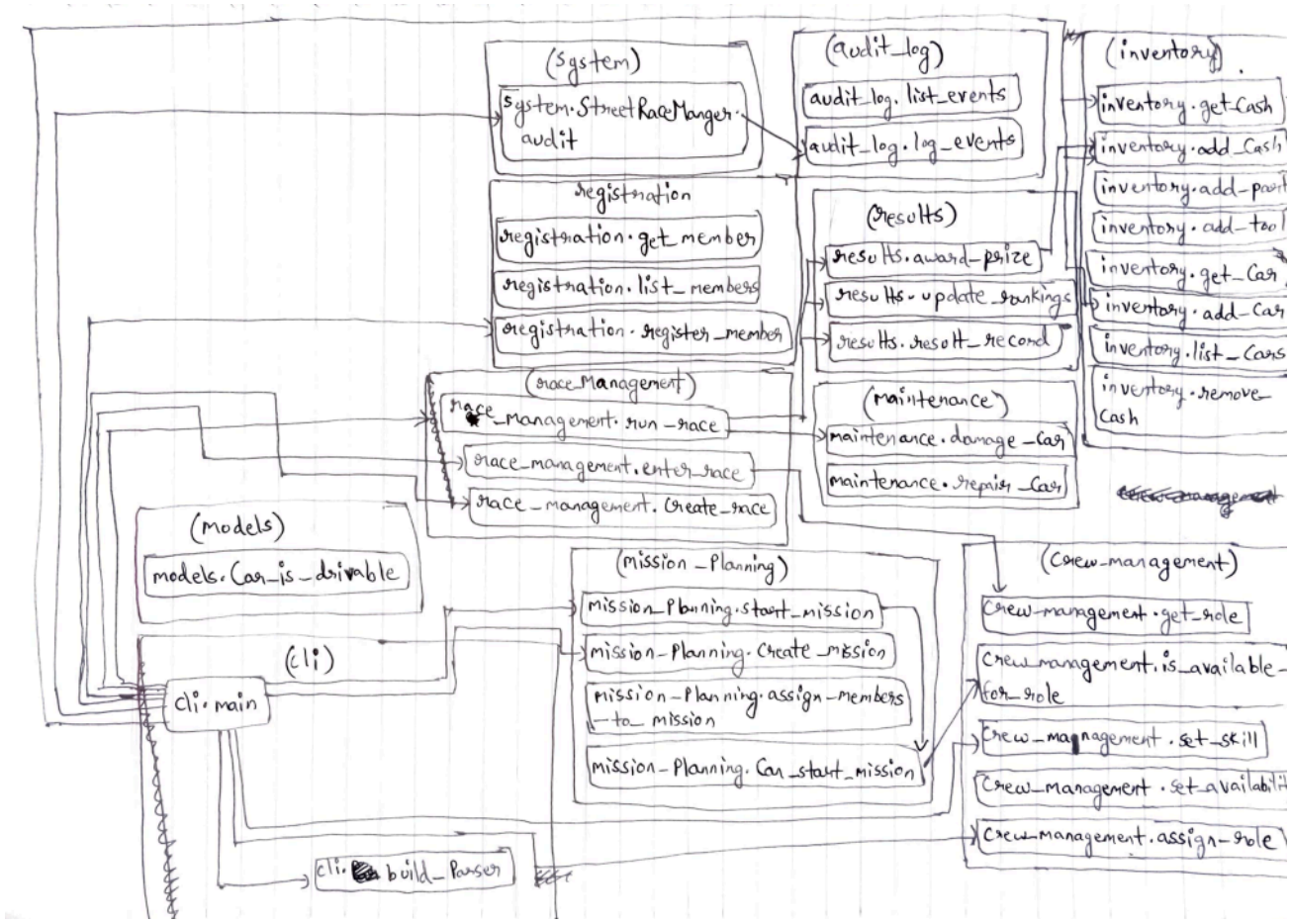
Diagram: